

Optimizer — Build Specification

Editable programme table · Reports consolidation · APT performance · Module-aware suggestions

What this document is

Four changes, specified against the code as it stands today. Part A makes the phase breakdown fully editable. Part B fixes the reports set — six modules currently produce no report at all. Part C explains why APT is slow and how to fix it. Part D makes the suggested questions follow the module the user is looking at. Each part is independent and can ship on its own.

Part A — Make the phase breakdown editable

Everything in the table stays as it is — the same phases, the same columns, the same generated values. The change is that the user can override any of it, and the programme re-plans around what they entered.

A1. How the table works today

The programme is generated, not typed. `schedule.py` derives every duration from the project's own quantities using $\text{work_days} = \text{quantity} \div (\text{output_per_day} \times \text{crew})$, links the activities with dependencies, and runs CPM to get dates. The user's only inputs today are start date, finish date, site setup days, masonry lag and concrete crew size — plus adding and removing whole tasks.

So the table is currently read-only output. Making it editable means adding a layer of user overrides that survive a re-plan, which is the core of this work.

A2. The data model — one new object

Add `task_overrides` to the schedule config, keyed by activity id. Every edit the user makes writes one entry here. The generator still runs normally; overrides are applied on top of its output.

```
task_overrides: {
  "t0_col_3": {
    "name": "Floor 3 columns – revised",
    "work_days": 6,
    "crew": 8,
    "cost": 284000,
    "start": "2026-11-04", // pins the start
    "finish": "2026-11-12", // pins the finish
    "order": 2
  }
}
```

Every field is optional. An absent field means “use the generated value”. This matters: if the user edits only the crew, the days must still recompute from the quantity — otherwise the edit silently breaks the arithmetic that makes this tool different from MS Project.

A3. Editing the task name and reordering

- **Name** — click the task text to edit inline; blur or Enter saves to `task_overrides[id].name`. Escape cancels. The generated name stays underneath, so a “reset to generated” action can always restore it.
- **Reorder** — drag handle on the left of each task row, drag within its phase only. Dropping writes an order integer to every task in that phase.
- **Reordering is display order, not schedule order.** This is the important distinction: the sequence work actually happens in is set by dependencies, not by row position. Dragging a task above another does not make it start earlier. Show a one-line note under the table saying so, or users will drag rows and wonder why the dates did not move.
- If the user wants a task to actually happen earlier, they pin its start date (A5) or change what it waits for — those are the two levers that move dates.

A4. How adding a task works today — and why “Survey” landed third

You added “Survey” to Pre-construction and it appeared third, after Project start and Mobilisation. That is not arbitrary — here is the exact mechanism.

- The Add-task form has a **Starts after** dropdown. Left blank, the new task is linked to the project start milestone — `Dep(after or "start")` in `apply_task_edits()`.
- So Survey has no real predecessor and its earliest start is the project start date — the same date as Project start and Mobilisation.
- The table sorts each phase's tasks by start date. Three tasks now share the same start date, and the sort is stable, so ties keep the order the tasks were created in.
- Generated tasks are built first; user-added tasks are appended last. Survey is therefore last among the tasks sharing that date — third — while Site clearing, which waits for Mobilisation and starts ten days later, sorts below it.

What to change here

Add the new task to the phase's display order at the position the user is looking at, rather than letting date-ties decide. Two concrete improvements: default the **Starts after** dropdown to the last task in the chosen phase instead of blank, so a new task chains onto the phase by default rather than jumping to day zero; and let the user drop the new row where they want it, writing an **order** value like any other reorder. Keep the blank option available and label it clearly as “Project start”.

A5. Editable start and finish dates

Replace the two date cells with date inputs carrying a calendar button — the same `<Input type="date">` already used for the programme start and finish fields, so the control is consistent with the rest of the app.

- Setting a date **pins** that task: CPM treats it as a constraint rather than a computed value. In scheduling terms this is a “start no earlier than” / “finish no earlier than” constraint.
- A pinned task shows a small pin marker so it is obvious which dates are the user's and which are the engine's. Clearing the field unpins it and the generated date returns.
- Pinning one task moves everything downstream of it, exactly as a real delay would. That is the point — this is how the user models the here-and-there slippage that a generated plan cannot know about.
- If a pinned date is **earlier** than the dependencies allow, do not silently accept it. Show the conflict on the row: which predecessor blocks it and by how many days.

The one hard limit

IS 456 curing and prop-removal lags stay non-negotiable. If a pinned date would strike props early, the app must refuse it and say why — this is the same guard `audit_safety()` already applies to generated lags, and it must apply to user-entered dates too. Every other constraint is advisory; this one is not.

A6. Editable days, crew and cost

Column	On edit	What recomputes
Days	Overrides the derived duration for that task.	All downstream dates. The implied output rate is shown as a hint so the user can see what productivity they just assumed.
Crew	Changes people on that task only.	Days recompute from $\text{quantity} \div (\text{output} \times \text{crew})$ — unless Days was also overridden, in which case the user's Days wins and only labour cost moves.
Cost	Overrides the computed cost for that task.	Phase cost, project total and the cash-flow S-curve. The BOQ is not touched — show the variance line that already exists under the table.

Precedence, stated once and applied everywhere: **a pinned date beats an edited Days value, which beats an edited Crew value, which beats the generated figure**. Show this as a one-line legend under the table so the behaviour is never a surprise.

A7. Defaults — every column populated on first load

Nothing should ever render blank or ask the user to fill it in before the table is usable. On generation every task already carries a name, phase, dates, days, crew and cost; the override layer starts empty and the table shows generated values throughout. Add a **Reset** action per row and per phase that clears overrides back to generated, and a single “Reset all edits” in the section header.

- Show edited cells with a subtle marker — a left border or a dot — so at a glance the user can see which numbers are theirs and which came from the engine.
- Keep a count in the section header: “6 tasks edited”, with the reset action beside it.
- Overrides persist with the project and travel with a saved version, so a comparison between two revisions reflects the edits.

Part B — Reports: what exists, what is missing, what to merge

B1. What exists today

Twelve PDF reports plus one Excel workbook, all defined in `reports.py` and listed in `ReportsModule.jsx`:

Report	Covers	Verdict
Executive Summary	Cross-module one-pager	Keep — becomes the parent of the merged set
IS / NBC Engineering Summary	All engineering modules	Keep — extend with the new modules
Structural Design Basis	Loads, seismic, foundation, mix, grid	Keep
Water & Sanitation	Demand, sump/OHT, STP, storm, RWH	Merge with Utility Report
Utility Report	Water, tanks, STP/WTP, RWH, plant rooms	Merge into Water & Sanitation — same numbers
Fire & Life Safety	NBC Part 4 checks	Keep
Accessibility Compliance	NBC Part 3 / RPwD	Merge into Compliance Report
BOQ Report	Material, labour, equipment	Merge with Quantity Report
Quantity Report	Thumb-rule ratios and quantities	Merge into BOQ — quantities are the unpriced half
Cost Report	Cost heads, per flat, per m ²	Keep
Parking Report	Required vs provided, ramps	Keep
Compliance Report	Rule-by-rule pass/fail	Keep — absorb Accessibility
BOQ Workbook (Excel)	Materials, labour, equipment sheets	Keep

B2. Modules with no report at all

These all ship in the app and none of them can be exported. This is the real gap.

Module	Menu location	Report to add
Programme	Cost & Programme	Construction Programme Report — phase table, critical path, floor cycle, IS 456 safety basis, cash-flow S-curve, resource histogram
Feasibility & ROI	Cost & Programme	Feasibility & ROI Report — revenue, cost, margin, ROI, IRR, payback, break-even, monthly cash flow
Setbacks & Controls	Site	Fold into Compliance — setbacks, FAR, height and coverage limits belong with the rule checks
Embodied Carbon (m13)	Engineering	Into Engineering Summary + a new Sustainability Report with the green rating
Plantation Plan (m14)	Engineering	Into the same Sustainability Report
Optimisers	BOQ / Cost / Planning	Optimisation Findings — current vs best vs change required, across all optimisers
GIS Intelligence	Site	Site Analysis Report — terrain, flood, access, sun path, suitability
Apartment Planning	Design	Into Executive Summary — unit mix and floor plate table

B3. The recommended final set — 11 reports

Down from 12 scattered ones, but covering every module instead of two thirds of them.

#	Report	Built from
1	Executive Summary	every module — the one a client reads

#	Report	Built from
2	Site Analysis	GIS: terrain, flood, access, sun, suitability
3	Compliance	rule checks + setbacks & controls + accessibility
4	IS / NBC Engineering Summary	all 14 engineering modules including carbon and trees
5	Structural Design Basis	loads, seismic, foundation, mix design, column grid, beam layout
6	Water & Sanitation	demand, sump/OHT, STP, storm, RWH, plant rooms
7	Fire & Life Safety	NBC Part 4
8	Sustainability	green rating, embodied carbon, plantation plan, solar potential
9	BOQ & Quantities	quantities, materials, labour, equipment — PDF and Excel
10	Cost & Feasibility	cost heads, per flat, per m ² , revenue, ROI, IRR, break-even
11	Construction Programme	phases, critical path, floor cycle, cash flow, resources

Parking and Optimisation findings fold into Compliance and Cost respectively rather than standing alone — a two-page PDF nobody opens is worse than a section in a report they already read. Keep every report downloadable individually and add a **Download all** that produces one merged PDF with a contents page.

Part C — Make APT fast

A “hi” currently costs the same as a hard engineering question. Here is exactly why, from the code.

C1. What happens on every single message

- `engine.analyse(proj)` — recomputes areas, quantities, BOQ, cost, parking, utilities and compliance from scratch.
- `englib.analyse_engineering(proj, an)` — recomputes all fourteen engineering modules.
- `aptlib.build(...)` — assembles the context, which internally calls `plan_schedule()` for the programme and the optimisers for their findings.
- The whole payload is serialised and sent with a long system prompt to the model — every time, regardless of what was asked.

So “hi” triggers a full re-analysis of the project plus a maximum-size prompt. The model is not the bottleneck; the work done before the model is called is.

C2. The five fixes, in order of payoff

Fix	What to do	Effect
1. Cache the analysis	Cache <code>analyse()</code> and <code>analyse_engineering()</code> per project, keyed on the project's <code>updated_at</code> . Recompute only when the project actually changed, not per message.	Removes nearly all server-side latency on follow-up messages in a conversation.
2. Intent gate	Classify the incoming message before building context. Greetings, thanks and “what can you do” answer from a fixed short path with no project context at all. Only questions that reference project data get the full payload.	“hi” returns instantly and costs almost nothing.
3. Tiered context	Three sizes: none for chit-chat, light (identity, headline metrics, failing rules) for general questions, full only when the question names a module, a number or a clause. Route on keywords first, not a model call.	Cuts average prompt size sharply without losing depth where it matters.
4. Stream the response	Server-sent events from the endpoint, render tokens as they arrive in <code>AptPanel.jsx</code> .	Time-to-first-word drops to under a second; the wait stops feeling like a hang.
5. Route the model	Send light-tier questions to the fast model in the existing fallback chain and keep the strong model for derivations and clause work.	Lower cost and latency on the majority of messages.

Keep the citation guard

None of these optimisations may skip `citations.verify()`. A fast answer with an invented IS clause is worse than a slow correct one. The guard runs on the returned text and costs nothing measurable.

Part D — Suggested questions that follow the module

Today GET /ai/chat/suggestions returns the same four questions everywhere — a failing compliance rule, the largest BOQ line, the programme finish date, and a fixed seismic question. They are good questions, but they do not change when the user moves from Programme to Parking.

D1. The change

- Add a ?module= parameter to the suggestions endpoint. AptPanel already knows the active module — pass it through and refetch when it changes.
- Build one small generator per module, each producing three or four questions from that module's own computed values. Fall back to today's general set when a module has nothing notable.
- Regenerate when the panel opens, not on every render, and cache per module per project version.

D2. What each module should ask

Module	Suggested questions drawn from that module's data
Plot & Site	Why is the buildable area smaller than the plot? · What is the setback on the north edge? · How much FAR is unused?
Setbacks & Controls	Which control is binding the height here? · What would a wider access road allow? · Which limit has least headroom?
GIS Intelligence	What does the flood risk score mean for foundations? · Which edge gets the best morning sun? · How does the slope affect excavation?
Apartment Planning	Why does this mix give this carpet ratio? · What happens to units if the plate grows 5%? · Which unit type earns most per m²?
Parking	Why is the requirement this number? · Where is the shortfall? · Which NBC clause sets the ECS rate?
Calculations	Which input drives this result most? · Show this step by step. · What tolerance applies here?
IS/NBC Engineering	Show the base shear derivation step by step. · Why this foundation type? · What drives the embodied carbon total?
BOQ & Quantities	Which line is the largest and why? · How was the steel quantity derived? · Where is the waste concentrated?
Cost Estimation	What is the cost per flat made of? · Which head moved most since the last revision? · Where is the biggest saving?
Programme	What is on the critical path? · Why can the floor cycle not go below this? · Which phase has most float?
Feasibility & ROI	What sale rate breaks even? · What drives the IRR most? · How does a six-month delay affect payback?
Compliance	Why does this rule fail? · What is the smallest change that passes? · Which clause governs this?
Reports	Which report has the client-facing summary? · What is in the engineering summary? · How current are these numbers?
Versions & Team	What changed between these two revisions? · Which change moved the cost most? · Did compliance improve?

Write them as questions built from the project's real numbers wherever possible — “Bricks are the largest line at ■5.5 Cr, what is driving it?” lands better than a generic “What is the largest cost?” The existing endpoint already does this well; the work is extending the pattern per module, not inventing it.

Build order and acceptance

Step	Scope	Done when
1	APT intent gate + analysis cache (Part C, fixes 1–3)	“hi” answers in under a second; a project question still answers fully with citations verified.
2	Module-aware suggestions (Part D)	Switching module changes the suggested questions; no module shows another module's questions.
3	Editable name and reorder (A2–A4)	Names persist across a re-plan; drag reorders within a phase; the display-order note is visible.

Step	Scope	Done when
4	Editable dates (A5)	A pinned date moves downstream tasks; an impossible pin is refused with the blocking predecessor named; IS 456 lags cannot be violated.
5	Editable days, crew, cost (A6–A7)	Precedence legend matches behaviour; edited cells are marked; reset restores generated values.
6	Reports consolidation (Part B)	Eleven reports, every module represented, Download-all produces one merged PDF.
7	Streaming + model routing (Part C, fixes 4–5)	First word appears under a second on a full-context question.

Steps 1 and 2 are the smallest and most visible — APT feels like a different product after them. Steps 3 to 5 are one continuous piece of work on the programme table and should ship together. Step 6 is independent and can run in parallel.